

MODUL 6 · TYPESCRIPT DALAM NEXT.JS

Tipe Dasar pada TypeScript



string, number, boolean, array, any, union, null dan undefined

Tipe Primitif

1. **string**
   Teks: judul artikel, nama, email

2. **number**
   Angka — bilangan bulat maupun desimal sama-sama number (tidak ada int atau float)

3. **boolean**
   true atau false: status aktif, sudah dibaca, dan sebagainya

Tipe Array

Kumpulan nilai dengan tipe yang sama.

1. **Tulis tipe elemen diikuti []**
   number[] artinya array yang berisi angka saja

2. **Bentuk lain: Array<number>**
   Artinya sama, tinggal pilih salah satu dan usahakan konsisten

3. **Menambah elemen bertipe lain akan ditolak**
   Array string tidak bisa diisi angka

Contoh Tipe Array

Dua penulisan array, dan satu kesalahan saat menambah elemen.

array.ts

```
let angka: number[] = [1, 2, 3];
let nama: Array<string> = ["Ani", "Budi"];

angka.push("4");
// Error: Argument of type 'string' is not assignable
// to parameter of type 'number'.
```

Type Inference

TypeScript sering bisa menebak tipe dari nilai awalnya.

1. **Variabel dengan nilai awal**
   let kota = "Bandung" otomatis bertipe string sehingga tidak perlu anotasi

2. **Parameter fungsi selalu kita tulis**
   TypeScript tidak bisa menebak dari mana data akan datang

3. **Return type biasanya cukup dibiarkan**
   Hasil kali dua number ditebak sebagai number

Contoh *Type Inference*

Tidak ada anotasi di variabel, tapi tipenya tetap dijaga.

inference.ts

```ts
let kota = "Bandung";
kota = 5;
// Error: Type 'number' is not assignable to type 'string'.

function kali(a: number, b: number) {
  return a * b;
}

const hasil = kali(2, 3);
hasil.toUpperCase();
// Error: Property 'toUpperCase' does not exist on type 'number'.
```

any: *Bypass* Pengecekan

Jalan pintas yang membuat TypeScript berhenti membantu.

1. **any = boleh apa saja**
   Nilai bertipe any tidak dicek: boleh dipanggil, diisi, diakses properti apa pun

2. **any bisa muncul tanpa kita tulis**
   Parameter tanpa tipe dianggap any. Opsi *noImplicitAny* (bagian dari strict) menjadikannya error

3. **Aturan di course ini: hindari any**
   Tulis tipe yang benar; any hanya mengembalikan risiko JavaScript

Bahayanya any

Editor diam saja, padahal kode ini pasti gagal saat dijalankan.

any.ts

```
let data: any = "halo";

data.toFixed();
// editor diam saja — crash saat dijalankan:
// data adalah string, bukan number

data = 42;
data.toUpperCase();
// editor diam saja — crash saat dijalankan
```

Union Type: Lebih dari Satu Kemungkinan

Gunakan | untuk mengatakan "salah satu dari tipe ini".

1. **number | string**
   Nilainya boleh angka, boleh teks, misalnya ID yang bisa berupa 101 atau "101"

2. **Hanya operasi yang valid untuk semua anggota**
   Untuk memakai fungsi khusus string, cek dulu dengan typeof

3. **Bisa digabung dengan null dan undefined**
   string | null artinya teks, atau kosong

Contoh *Union Type*

Cek tipe dengan typeof sebelum memakai fungsi khusus string.

union.ts

```ts
function tampilkanId(id: number | string) {
  id.toUpperCase();
  // Error: Property 'toUpperCase' does not exist
  // on type 'string | number'.

  if (typeof id === "string") {
    id.toUpperCase(); // aman: id pasti string
  }
}
```

null, undefined, dan Parameter Opsional

Sumber error "Cannot read properties of undefined" yang paling sering.

1. **Dengan strict, null dan undefined bukan anggota tipe lain**
   string hanya berisi teks; yang bisa kosong harus ditulis string | null

2. **Tanda ? membuat parameter opsional**
   gelar?: string artinya string atau undefined

3. **Cek dulu sebelum dipakai**
   Dengan if, atau optional chaining ?.

Contoh null dan undefined

TypeScript memaksa kita menangani kemungkinan kosong sebelum bug terjadi.

null.ts

```
function panjang(teks: string | null) {
  return teks.length;
  // Error: 'teks' is possibly 'null'.
}

function panjangAman(teks: string | null) {
  if (teks === null) return 0;
  return teks.length; // aman: teks pasti string
}
```

Contoh Parameter Opsional

Tanda ? membuat gelar bisa undefined, pakai ?. atau if sebelum mengaksesnya.

opsional.ts

```ts
function sapa(nama: string, gelar?: string) {
  return gelar.toUpperCase();
  // Error: 'gelar' is possibly 'undefined'.
}

function sapaAman(nama: string, gelar?: string) {
  return gelar?.toUpperCase(); // aman: hasilnya string | undefined
}
```